

Lecture-3

HTML + CSS + JS: Foundations to DOM

Tags, styling methods, Browser vs Node.js, this, var/let/const, and live DOM manipulation



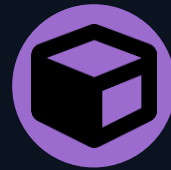
HTML Tags



CSS: 3 ways



Browser vs Node



this / var / let



DOM Demo

HTML — the structure layer

Every page starts with a small set of tags that define its skeleton



`<!DOCTYPE html>`

Tells the browser: parse this as HTML5

`<html> <head> <body>`

Root, metadata, and visible content

`<title> <meta> <link>`

Page title, charset/viewport, external CSS

`<h1>...<h6>, <p>, <a>, <img>`

Headings, paragraphs, links, images

`<div>, <span>`

Generic block / inline containers

`<ul>/<ol>/<li>, <script>`

Lists, and JS inclusion

```
<!DOCTYPE html>
<html>
  <head><title>My Page</title></head>
  <body>
    <h1>Hello World</h1>
    <p>This is a paragraph.</p>
  </body>
</html>
```

Three ways to bring CSS into HTML

Same styling goal, three different points of attachment



Inline

```
<h1 style="color:red">
  Hi
</h1>
```

Written directly on the tag via the style attribute. Highest specificity, hardest to maintain.



Internal

```
<style>
  h1 { color: red; }
</style>
```

A `<style>` block inside `<head>`. Scoped to this one HTML file only.



External

```
<link rel="stylesheet"
      href="style.css">
```

A separate .css file linked in `<head>`. Reusable across many pages — the recommended approach.

JavaScript in the Browser vs Node.js

Same language, two very different environments ("runtimes") around it



Browser

- Has `window`, `document`, DOM APIs
- Can do `alert()`, `localStorage`
- No filesystem, no `require()`
- Sandboxed for security



Node.js

- Has `global`, `module`, `process`
- Can do file I/O (`fs`), servers (`http`)
- No `window`, no `document`, no DOM
- Full system access

Same code, different output — Example 1

`console.log(this)` at the top level of a file

```
console.log(this);
```



In the Browser

`this` → the Window object
(global object of the browser tab)



In Node.js

`this` → `module.exports`
(an empty object {}, not global)

Same code, different output — Example 2

Referencing globals and DOM-only APIs

Expression	Browser	Node.js
<code>window.alert('hi')</code>	Shows an alert popup	ReferenceError: window is not defined
<code>document.title</code>	Reads the page's <title>	ReferenceError: document is not defined
<code>require('fs')</code>	ReferenceError: require is not defined (in plain <script>)	Reads/writes files on disk normally
<code>typeof global</code>	"undefined"	"object" — Node's global object

The 'this' keyword

`this` isn't fixed — it depends on HOW a function is called, not where it's written



Regular function (browser)

```
function show(){ console.log(this); }  
show();
```

this → window (called with no owner)

Object method

```
const user = { name: 'Ana',  
  greet(){ console.log(this.name); } };  
user.greet();
```

this → user (the object before the dot)

Arrow function

```
const obj = { greet: () => {  
  console.log(this); } };  
obj.greet();
```

this → outer scope's this (arrows don't bind their own)

var vs let vs const — hoisting behavior

Same access-before-declare pattern, three different results



var

```
console.log(a);  
var a = 5;
```

→ **undefined**

var is hoisted AND initialized to undefined at the top of its scope.



let

```
console.log(b);  
let b = 5;
```

→ **ReferenceError**

let is hoisted but stays in the "Temporal Dead Zone" until its line runs.



const

```
console.log(c);  
const c = 5;
```

→ **ReferenceError**

Same TDZ rule as let — plus it can never be reassigned after init.

const: can't reassign, CAN mutate

const locks the binding, not the contents of an object/array

```
const num = 5;  
num = 10;  
// TypeError: Assignment to  
// constant variable.
```

```
const user = { name: "Ana" };  
user.name = "Priya"; // ✓ works  
user.age = 25;       // ✓ works  
// user = {} ✗ still an error
```

const only prevents re-pointing the variable name to a new value — it never freezes what's inside an object or array. Use `Object.freeze()` if you need true immutability.

Live demo: changing an <h1> with JavaScript

Selecting a real DOM node and mutating its style properties

```
<h1 id="title">Hello World</h1>

<script>
  const heading = document.getElementById("title");
  heading.style.backgroundColor = "#111A2E";
  heading.style.color = "#D9A94A";
</script>
```

1. `document.getElementById("title")` finds the real DOM node.
2. `.style.backgroundColor` / `.style.color` directly edits inline CSS on that node.
3. The browser re-runs Layout → Paint for just this element — no page reload needed.

Recap

What we covered today



HTML tags

The structural skeleton of every page



3 ways to add CSS

Inline, internal <style>, external .css



Browser vs Node.js

Same language, different globals & capabilities



this keyword

Depends on how a function is called



var vs let vs const

undefined vs Temporal Dead Zone



DOM manipulation

getElementById + .style to change live pages

Thank You !